

### Practical No-03

Name- Reeya Borikar  
Roll No- 16  
Registration No- 23030072

**AIM-** Develop GUI- Image Transformation(Contrast Stretching+ Intensity Level Slicing+ Bit Plane Slicing)

#### CODE-

```
function Practical_3_image_transformation_gui
% GUI for Image Transformations: Contrast Stretching, Level Slicing, Bit Plane Slicing

% --- Constants and Styling ---
C_BG = [0.94 0.94 0.94]; % Light Gray Background
C_BLUE = [0 0.48 0.8]; % MATLAB Blue for main actions
C_TEXT = [0.1 0.1 0.1]; % Dark Gray Text
C_PANEL = [1 1 1]; % White Panel Background

% --- UI Figure Setup ---
fig = uifigure('Name','Image Transformation Tools','Position',[100 100 950 600],...
    'Color', C_BG, 'Resize', 'off');

% --- Variables to hold image data ---
img = []; % uint8 grayscale (original)
filteredImg = []; % uint8 or logical (transformed)

% --- UI Component Definitions ---

% --- Control Panel (Left Side) ---
panelControls = uipanel(fig, 'Title', 'Controls & Parameters', ...
    'Position', [20 20 280 560], ...
    'BackgroundColor', C_PANEL, ...
    'ForegroundColor', C_TEXT, ...
    'FontWeight', 'bold', 'FontSize', 12);

% Load Image Button
uibutton(panelControls,'push','Text','Load Image',...
    'Position',[40 500 200 40],...
    'BackgroundColor', [0.85 0.85 0.85], ...
    'FontSize', 14, ...
    'ButtonPushedFcn',@(~,~) loadImage());

% Dropdown for selecting transformation mode
uilabel(panelControls,'Text','Select Transformation:',...
    'Position',[20 440 200 22], 'FontSize', 12, 'FontWeight', 'bold');
ddMode = uidropdown(panelControls,...
    'Items',{'Contrast Stretching', 'Intensity Level Slicing', 'Bit Plane Slicing'},...
    'Position',[20 410 240 30], ...
    'ValueChangedFcn', @(~,~) modeChanged(), ...
    'FontSize', 10);

% Apply Button
```

```

btnApply = uibutton(panelControls,'push','Text','Apply Transformation',...
    'Position',[40 30 200 40],...
    'BackgroundColor', C_BLUE, ...
    'FontColor', [1 1 1], ...
    'FontSize', 14, ...
    'FontWeight', 'bold', ...
    'ButtonPushedFcn',@(~,~) applyTransformation());

% Status label
lblStatus = uilabel(fig,'Text','Load an image to start.',...
    'Position',[30 5 400 22], 'HorizontalAlignment', 'left', ...
    'FontSize', 10, 'FontColor', C_TEXT);

% --- Parameter Panel (Dynamic) ---
panelParams = uipanel(panelControls, 'Title', 'Parameters', ...
    'Position', [20 80 240 310], ...
    'BackgroundColor', C_PANEL, ...
    'ForegroundColor', C_TEXT, ...
    'FontWeight', 'bold', 'FontSize', 12);

% --- Axes (Right Side) ---

% Axes to show original image
axOrig = uiaxes(fig,'Position',[320 20 290 560], 'Box','on', 'Color', 'w');
title(axOrig,'Original Image (Grayscale)','FontSize',14, 'FontWeight','bold', 'Color', C_TEXT);
axis(axOrig,'off');

% Axes to show transformed image
axTransformed = uiaxes(fig,'Position',[630 20 290 560], 'Box','on', 'Color', 'w');
title(axTransformed,'Transformed Image','FontSize',14, 'FontWeight','bold', 'Color', C_TEXT);
axis(axTransformed,'off');

% Initialize dynamic parameter controls
modeChanged();

% --- CALLBACK FUNCTIONS ---

function loadImage()
    [file,path] = uigetfile({'*.jpg;*.png;*.bmp;.tif','Image Files (*.jpg, *.png, *.bmp, *.tif)'});
    if isequal(file,0)
        lblStatus.Text = 'Image load cancelled.';
        return;
    end
    imgPath = fullfile(path,file);

    try
        temp_img = imread(imgPath);

        % Convert to grayscale (uint8)
        if size(temp_img,3)==3
            img = rgb2gray(temp_img);
        else

```

```

        img = temp_img;
    end

    % Ensure image is uint8
    if ~isa(img, 'uint8')
        img = im2uint8(img);
    end

    imshow(img, 'Parent', axOrig);
    lblStatus.Text = 'Image loaded successfully. Ready to transform.';
    cla(axTransformed);
    title(axTransformed, 'Transformed Image');

catch ME
    lblStatus.Text = ['Error loading image: ', ME.message];
end
end

function modeChanged()
    % Clear existing controls in the parameter panel
    delete(findobj(panelParams.Children));

    mode = ddMode.Value;

    switch mode
    case 'Contrast Stretching'
        uilabel(panelParams, 'Text', 'Input Range [r1, r2] (0-255):', ...
            'Position', [20 260 200 20]);
        uieditfield(panelParams, 'numeric', 'Value', 50, 'Position', [20 230 100 22], 'Tag', 'r1');
        uieditfield(panelParams, 'numeric', 'Value', 150, 'Position', [140 230 100 22], 'Tag', 'r2');

        uilabel(panelParams, 'Text', 'Output Range [s1, s2] (0-255):', ...
            'Position', [20 180 200 20]);
        uieditfield(panelParams, 'numeric', 'Value', 0, 'Position', [20 150 100 22], 'Tag', 's1');
        uieditfield(panelParams, 'numeric', 'Value', 255, 'Position', [140 150 100 22], 'Tag', 's2');

    case 'Intensity Level Slicing'
        uilabel(panelParams, 'Text', 'Lower Threshold (A, 0-255):', ...
            'Position', [20 260 200 20]);
        uieditfield(panelParams, 'numeric', 'Value', 100, 'Position', [20 230 100 22], 'Tag', 'A');

        uilabel(panelParams, 'Text', 'Upper Threshold (B, 0-255):', ...
            'Position', [20 180 200 20]);
        uieditfield(panelParams, 'numeric', 'Value', 200, 'Position', [20 150 100 22], 'Tag', 'B');

        uilabel(panelParams, 'Text', 'Slice Highlight Value:', ...
            'Position', [20 100 200 20]);
        ddHighlight = uidropdown(panelParams, ...
            'Items', {'Set to White (255)', 'Keep Original'}, ...
            'ItemsData', {true, false}, ...
            'Value', true, ...
            'Position', [20 70 200 30], 'Tag', 'highlight');
    end
end

```

```

case 'Bit Plane Slicing'
    uilabel(panelParams, 'Text', 'Select Bit Plane (1=LSB, 8=MSB):', ...
        'Position', [20 260 200 20]);
    ddBit = uidropdown(panelParams, ...
        'Items', {'Plane 8 (MSB)', 'Plane 7', 'Plane 6', 'Plane 5', 'Plane 4', 'Plane 3', 'Plane 2', 'Plane 1
(LSB)'}, ...
        'ItemsData', {8, 7, 6, 5, 4, 3, 2, 1}, ...
        'Value', 8, ...
        'Position', [20 230 200 30], 'Tag', 'bitplane');
end
end

```

```

function applyTransformation()

```

```

    if isempty(img)
        lblStatus.Text = 'Please load an image first.';
        return;
    end

```

```

    mode = ddMode.Value;

```

```

    try

```

```

        switch mode

```

```

            case 'Contrast Stretching'

```

```

                r1 = findobj(panelParams.Children, 'Tag', 'r1').Value;
                r2 = findobj(panelParams.Children, 'Tag', 'r2').Value;
                s1 = findobj(panelParams.Children, 'Tag', 's1').Value;
                s2 = findobj(panelParams.Children, 'Tag', 's2').Value;

```

```

                % Clamp values to valid range (0-255) and ensure r1 < r2, s1 < s2

```

```

                r1 = max(0, min(255, r1));
                r2 = max(0, min(255, r2));
                s1 = max(0, min(255, s1));
                s2 = max(0, min(255, s2));

```

```

                if r1 >= r2 || s1 >= s2

```

```

                    error('Input and Output limits must be strictly increasing (r1 < r2 and s1 < s2).');

```

```

                end

```

```

                % Transformation logic

```

```

                imgDouble = double(img);
                filteredImg = zeros(size(imgDouble));

```

```

                % Zone 1: r < r1 (slope s1/r1)

```

```

                idx1 = imgDouble < r1;
                filteredImg(idx1) = imgDouble(idx1) * (s1 / r1);

```

```

                % Zone 2: r1 <= r <= r2 (slope (s2-s1)/(r2-r1))

```

```

                idx2 = (imgDouble >= r1) & (imgDouble <= r2);
                m = (s2 - s1) / (r2 - r1);
                c = s1 - m * r1;
                filteredImg(idx2) = imgDouble(idx2) * m + c;

```

```

% Zone 3:  $r > r_2$  (slope  $(255-s_2)/(255-r_2)$ )
idx3 = imgDouble > r2;
m_high = (255 - s2) / (255 - r2);
c_high = s2 - m_high * r2;
filteredImg(idx3) = imgDouble(idx3) * m_high + c_high;

filteredImg = uint8(filteredImg);

```

#### case 'Intensity Level Slicing'

```

A = findobj(panelParams.Children, 'Tag', 'A').Value;
B = findobj(panelParams.Children, 'Tag', 'B').Value;
highlight = findobj(panelParams.Children, 'Tag', 'highlight').Value;

```

```

% Clamp values and ensure  $A < B$ 

```

```

A = max(0, min(255, A));
B = max(0, min(255, B));

```

```

if A >= B

```

```

    error('Lower Threshold (A) must be less than Upper Threshold (B).');

```

```

end

```

```

filteredImg = img; % Start with the original image

```

```

slice_mask = (img >= A) & (img <= B);

```

```

if highlight % Highlight slice (set to white)

```

```

    % All pixels outside the slice are set to black (0)

```

```

    filteredImg(~slice_mask) = 0;

```

```

    % All pixels inside the slice are set to white (255)

```

```

    filteredImg(slice_mask) = 255;

```

```

else % Keep original, set outside to black

```

```

    % All pixels outside the slice are set to black (0)

```

```

    filteredImg(~slice_mask) = 0;

```

```

    % Pixels inside the slice keep their original value (which they already do by starting with img)

```

```

end

```

```

filteredImg = uint8(filteredImg);

```

#### case 'Bit Plane Slicing'

```

bit_plane = findobj(panelParams.Children, 'Tag', 'bitplane').Value;

```

```

%  $2^{(\text{bit\_plane} - 1)}$  gives the weight of the bit

```

```

% bitget extracts the bit (0 or 1)

```

```

bit_image = bitget(img, bit_plane);

```

```

% Multiply by 255 to display the 0s and 1s as black and white

```

```

filteredImg = uint8(bit_image * 255);

```

```

end

```

```

catch ME

```

```

    lblStatus.Text = ['Error applying transformation: ', ME.message];

```

```

    cla(axTransformed);

```

```

    return;

```

```

end

```

```
% --- Display transformed image ---  
imshow(filteredImg, 'Parent', axTransformed);
```

```
lblStatus.Text = ['Applied ', mode, ' successfully.'];  
title(axTransformed,[mode, ' Result']);
```

```
end  
end
```

## OUTPUT-

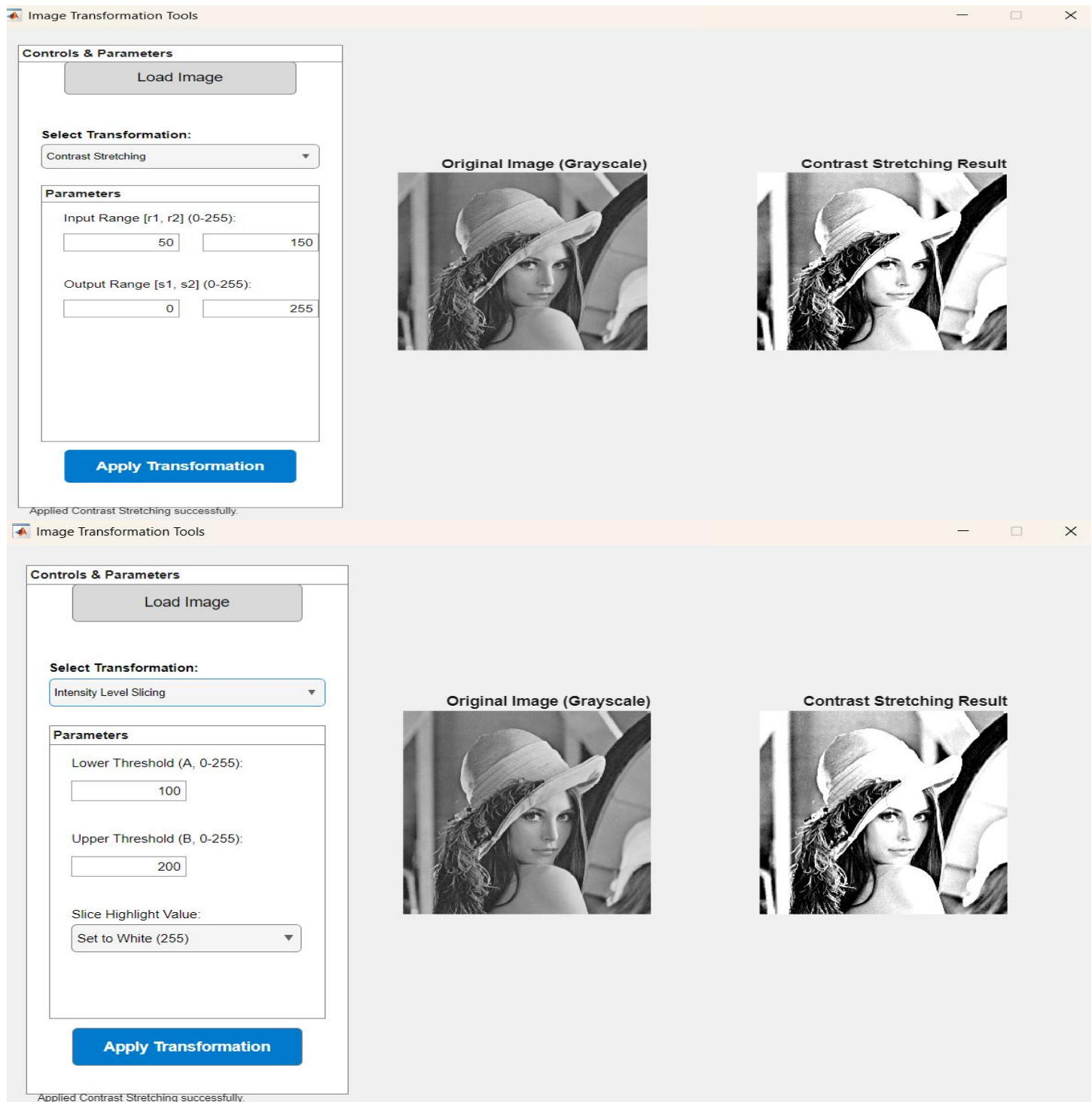


Image Transformation Tools

Controls & Parameters

Load Image

Select Transformation:

Bit Plane Slicing

Parameters

Select Bit Plane (1=LSB, 8=MSB):

Plane 8 (MSB)

Apply Transformation

Original Image (Grayscale)



Contrast Stretching Result



Applied Contrast Stretching successfully.